

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XII
PENERAPAN PRINSIP SOLID PADA SISTEM INFORMASI AKADEMIK
(SIKAD)



Disusun Oleh:
Ni Putu Haruka Ikeda (105224039)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Sistem Informasi Akademik (SIKAD) merupakan aplikasi yang digunakan untuk mendukung berbagai kegiatan akademik di lingkungan perguruan tinggi, salah satunya adalah proses pengisian Kartu Rencana Studi (KRS). Seiring meningkatnya jumlah pengguna dan kebutuhan sistem, kualitas desain perangkat lunak menjadi faktor penting untuk menjaga performa, kemudahan pemeliharaan, serta kemampuan sistem dalam beradaptasi terhadap perubahan.

Pada studi kasus yang diberikan, sistem SIKAD mengalami berbagai permasalahan arsitektur perangkat lunak. Kelas SistemKRSManager memiliki terlalu banyak tanggung jawab, mulai dari validasi prasyarat mata kuliah, perhitungan UKT, pembuatan dokumen PDF, hingga penyimpanan data ke database. Kondisi ini menyebabkan sistem menjadi sulit dipelihara dan rentan mengalami kerusakan ketika dilakukan perubahan pada salah satu bagian program. Selain itu, terdapat penggunaan antarmuka yang tidak sesuai kebutuhan, ketergantungan langsung terhadap implementasi database tertentu, serta logika perhitungan UKT yang dipenuhi banyak percabangan sehingga sulit dikembangkan.

Untuk mengatasi permasalahan tersebut, dilakukan perancangan ulang sistem dengan menerapkan prinsip-prinsip SOLID pada Pemrograman Berorientasi Objek (Object-Oriented Programming). Prinsip SOLID terdiri dari Single Responsibility Principle (SRP), Open Closed Principle (OCP), Liskov Substitution Principle (LSP), Interface Segregation Principle (ISP), dan Dependency Inversion Principle (DIP). Penerapan prinsip-prinsip tersebut bertujuan untuk menghasilkan struktur kode yang lebih modular, mudah dikembangkan, mudah dipelihara, serta mampu beradaptasi terhadap perubahan kebutuhan sistem di masa mendatang.

Pada praktikum ini dilakukan implementasi rancangan ulang sistem KRS dengan memisahkan tanggung jawab setiap kelas, menggunakan strategi perhitungan UKT yang fleksibel, memisahkan antarmuka berdasarkan kebutuhan, serta menerapkan abstraksi pada mekanisme penyimpanan data. Hasil implementasi diharapkan dapat menghasilkan sistem yang lebih stabil, mudah dikembangkan, dan siap mendukung migrasi database maupun penambahan fitur baru tanpa mengganggu fungsi yang telah ada.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	<i>mahasiswa</i>	String	Menyimpan nama mahasiswa yang melakukan pengisian KRS
2	<i>mataKuliah</i>	String	Menyimpan nama mata kuliah yang dipilih
3	<i>data</i>	String	Menyimpan informasi KRS yang akan dicetak atau disimpan
4	<i>database</i>	Database	Menyimpan objek database yang digunakan untuk penyimpanan data KRS
5	<i>validator</i>	KrsValidator	Objek untuk melakukan validasi prasyarat mata kuliah
6	<i>ukt</i>	UKTStrategy	Objek yang digunakan untuk menghitung UKT berdasarkan jalur mahasiswa
7	<i>pdf</i>	PDFGenerator	Objek yang digunakan untuk menghasilkan dokumen PDF KRS
8	<i>service</i>	KRSService	Objek yang menangani penyimpanan data KRS ke database
9	<i>teori</i>	MataKuliah	Objek mata kuliah jenis teori
10	<i>praktikum</i>	MataKuliahPraktikum	Objek mata kuliah jenis praktikum
11	<i>kkn</i>	MataKuliah	Objek mata kuliah jenis KKN

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KRSService (Database database)	<i>constructor</i>	Menginisialisasi objek KRSService dengan database tertentu
2	saveKRS(String data)	Procedural	Menyimpan data KRS ke database
3	validasiPrasyarat()	Functional	Memvalidasi prasyarat mata kuliah mahasiswa
4	generatePDF()	Procedural	Mencetak data KRS ke dalam format PDF
5	hitungUKT()	Functional	Menghitung UKT berdasarkan jalur mahasiswa.
6	tampilkanInfo()	Procedural	Menampilkan informasi mata kuliah.
7	alokasiAsistenLab()	Procedural	Mengalokasikan asisten laboratorium untuk praktikum
8	cekPeralatanPraktikum()	Functional	Memeriksa ketersediaan peralatan praktikum

IV. Dokumentasi dan Pembahasan Code

4.1 Interface Database

```
package database;

public interface Database {
    void saveKRS(String data);
}
```

Interface Database digunakan sebagai abstraksi untuk proses penyimpanan data KRS. Dengan adanya interface ini, sistem tidak bergantung pada implementasi database tertentu. Kelas lain cukup berinteraksi dengan interface Database tanpa mengetahui detail penyimpanan data yang digunakan.

4.2 Kelas sqlDatabase dan NosqlDatabase

```
//sqldatabase
package database;

public class sqlDatabase implements Database {

    @Override
    public void saveKRS(String data) {
        System.out.println("Data disimpan ke
MySQL : " + data);
    }
}

//sqlnodatabase
package database;

public class NosqlDatabase implements Database {

    @Override
    public void saveKRS(String data) {
        System.out.println("Data disimpan ke Cloud
NoSQL : " + data);
    }
}
```

Kedua kelas tersebut merupakan implementasi dari interface Database. Dengan desain ini, sistem dapat berpindah dari MySQL ke NoSQL tanpa mengubah kode pada KRSService. Implementasi ini menerapkan Dependency Inversion Principle (DIP), yaitu ketergantungan terhadap abstraksi, bukan terhadap implementasi konkret.

4.3 Kelas KrsValidator

```
package service;

public class KrsValidator {

    public boolean validasiPrasyarat(
```

```

        String mahasiswa,
        String matakuliah) {

        System.out.println(
            "Validasi prasyarat "
            + matakuliah
            + " milik "
            + mahasiswa);

        return true;
    }
}

```

Kelas KrsValidator bertanggung jawab untuk melakukan validasi prasyarat mata kuliah yang akan diambil mahasiswa. Pada sistem lama, fungsi validasi berada di dalam SistemKRSManger bersama banyak fungsi lainnya. Pada rancangan baru, fungsi validasi dipisahkan ke kelas tersendiri sehingga menerapkan Single Responsibility Principle (SRP).

4.4 Kelas PDFGenerator

```

package service;

public class PDFGenerator {

    public void generatePDF(String data) {
        System.out.println("Mencetak PDF :
" + data);
    }
}

```

Kelas PDFGenerator digunakan untuk menghasilkan dokumen KRS dalam bentuk PDF. Pemisahan ini membuat perubahan format dokumen tidak memengaruhi proses validasi maupun penyimpanan data. Hal ini juga merupakan penerapan Single Responsibility Principle (SRP).

4.5 Interface UKTStrategy

```

package ukt;

public interface UKTStrategy {
    double hitungUKT();
}

```

Interface UKTStrategy digunakan untuk mendefinisikan metode perhitungan UKT yang akan digunakan oleh berbagai jalur mahasiswa. Dengan pendekatan ini, sistem tidak lagi menggunakan banyak percabangan if-else yang sulit dipelihara.

4.6 Kelas UKTReguler

```
package ukt;

public class UKTReguler implements UKTStrategy
{
    @Override
    public double hitungUKT() {
        return 5000000;
    }
}
```

Kelas UKTReguler merupakan implementasi dari interface UKTStrategy yang digunakan untuk menghitung biaya UKT mahasiswa jalur reguler. Pada kelas ini, method `hitungUKT()` mengembalikan nilai UKT sebesar Rp5.000.000.

Pemisahan perhitungan UKT ke dalam kelas tersendiri membuat algoritma perhitungan jalur reguler tidak bercampur dengan jalur lainnya. Dengan demikian, apabila terjadi perubahan tarif UKT reguler, pengembang hanya perlu melakukan perubahan pada kelas UKTReguler tanpa memengaruhi kelas lain.

Selain itu, kelas ini mendukung penerapan **Open Closed Principle (OCP)** karena sistem dapat menambahkan jenis jalur baru tanpa memodifikasi kode yang sudah ada.

4.7 Kelas UKTkaryawan

```
package ukt;

public class UKTkaryawan implements UKTStrategy {
    @Override
    public double hitungUKT() {
        return 7000000;
    }
}
```

Kelas UKTkaryawan digunakan untuk menghitung biaya UKT mahasiswa jalur karyawan. Kelas ini mengimplementasikan interface UKTStrategy sehingga memiliki method `hitungUKT()` yang mengembalikan nilai UKT sebesar Rp7.000.000.

Dengan adanya kelas khusus untuk jalur karyawan, sistem menjadi lebih fleksibel dan mudah dipelihara. Perubahan aturan UKT jalur karyawan tidak akan memengaruhi perhitungan UKT pada jalur lainnya.

Penerapan strategi ini mengurangi penggunaan percabangan `if-else` yang berlebihan pada sistem lama dan membuat kode lebih mudah dikembangkan.

4.8 Kelas UKTInternasional

```
package ukt;

public class UKTInternasional implements UKTStrategy {

    @Override
    public double hitungUKT() {
        return 12000000;
    }
}
```

Kelas UKTInternasional digunakan untuk menangani perhitungan UKT mahasiswa internasional. Method hitungUKT() mengembalikan nilai UKT sebesar Rp12.000.000.

Pemisahan logika perhitungan ke dalam kelas tersendiri membuat proses pengelolaan tarif mahasiswa internasional menjadi lebih terstruktur. Jika di masa mendatang terdapat perubahan kebijakan biaya pendidikan mahasiswa internasional, maka perubahan hanya dilakukan pada kelas ini.

Pendekatan tersebut meningkatkan modularitas sistem dan mendukung prinsip **Open Closed Principle (OCP)**.

4.9 Kelas UKTbidikmisi

```
package ukt;

package ukt;

public class UKTbidikmisi implements UKTStrategy {

    @Override
    public double hitungUKT() {
        return 0;
    }
}
```

Kelas UKTbidikmisi digunakan untuk menghitung UKT mahasiswa penerima program Bidikmisi. Pada implementasi ini, method hitungUKT() mengembalikan nilai 0 karena mahasiswa Bidikmisi memperoleh bantuan biaya pendidikan.

Dengan memisahkan aturan perhitungan Bidikmisi ke dalam kelas tersendiri, sistem menjadi lebih mudah dipahami dan dipelihara. Setiap jenis jalur memiliki algoritma perhitungan yang berdiri sendiri sehingga tidak saling bergantung.

Hal ini membuat sistem lebih fleksibel ketika terjadi perubahan kebijakan beasiswa di masa mendatang.

4.9 Kelas UKTMBKM

```

package ukt;

public class UKTMBKM implements UKTStrategy {

    @Override
    public double hitungUKT() {
        return 3000000;
    }
}

```

Kelas UKTMBKM digunakan untuk menghitung UKT mahasiswa yang mengikuti program Merdeka Belajar Kampus Merdeka (MBKM). Method `hitungUKT()` mengembalikan nilai UKT sebesar Rp3.000.000 sesuai skenario yang digunakan pada program.

Kelas ini menjadi contoh bagaimana sistem dapat dikembangkan dengan mudah ketika kampus menambahkan jalur atau program baru. Pengembang cukup membuat kelas baru yang mengimplementasikan interface `UKTStrategy` tanpa perlu mengubah kode pada kelas yang telah ada sebelumnya.

Dengan demikian, sistem menjadi lebih siap menghadapi perubahan kebutuhan dan lebih mudah untuk dikembangkan di masa depan.

4.10 Interface MataKuliah dan Praktikum

```

//matakuliah
package matakuliah;

public interface MataKuliah {
    void tampilkanInfo();
}

//praktikum
package matakuliah;

public class MataKuliahPraktikum
    implements MataKuliah, Praktikum {

    @Override
    public void tampilkanInfo() {
        System.out.println("Mata Kuliah
Praktikum");
    }

    @Override
    public void alokasiAsistenLab() {
        System.out.println("Asisten Lab
dialokasikan");
    }

    @Override
    public void cekPeralatanPraktikum() {
        System.out.println("Peralatan dicek");
    }
}

```

Pada sistem lama, seluruh mata kuliah diwajibkan mengimplementasikan fungsi praktikum meskipun tidak membutuhkannya. Hal tersebut menyebabkan munculnya `UnsupportedOperationException`. Pada rancangan baru, antarmuka dipisahkan menjadi `MataKuliah` dan `Praktikum` sehingga hanya kelas yang membutuhkan fitur praktikum yang mengimplementasikannya. Pendekatan ini menerapkan Interface Segregation Principle (ISP).

4.11 Kelas `MataKuliahTeori`

```
package matakuliah;

public class MataKuliahTeori implements MataKuliah
{
    @Override
    public void tampilkanInfo() {
        System.out.println("Mata Kuliah Teori");
    }
}
```

Kelas `MataKuliahTeori` hanya mengimplementasikan fungsi yang relevan dengan mata kuliah teori tanpa dipaksa memiliki fungsi praktikum.

4.12 Kelas `MataKuliahPraktikum`

```
public class MataKuliahPraktikum implements MataKuliah,
Praktikum {
```

Kelas `MataKuliahPraktikum` mengimplementasikan kedua interface karena membutuhkan informasi mata kuliah sekaligus fitur praktikum seperti alokasi asisten laboratorium dan pengecekan peralatan.

4.13 Kelas `MataKuliahKKN`

```
package matakuliah;

public class MataKuliahKKN implements MataKuliah {

    @Override
    public void tampilkanInfo() {
        System.out.println("Mata Kuliah KKN");
    }
}
```

Kelas `MataKuliahKKN` hanya mengimplementasikan interface `MataKuliah` karena tidak memerlukan fitur praktikum. Desain ini menghindari implementasi metode yang tidak digunakan.

4.14 Kelas `KRSService`

```
package service;

import database.Database;
```

```

public class KRSService {
    private Database database;

    public KRSService(Database database) {
        this.database = database;
    }

    public void simpanKRS(String data) {
        database.saveKRS(data);
    }
}

```

Kelas KRSService bertugas untuk menyimpan data KRS ke database. Kelas ini tidak bergantung langsung pada MySQL maupun NoSQL, melainkan pada interface Database. Dengan demikian, proses migrasi database dapat dilakukan tanpa mengubah logika bisnis aplikasi. Desain ini merupakan penerapan Dependency Inversion Principle (DIP).

4.15 Kelas Main

```

package Main;

import database.*;
import service.*;
import ukt.*;
import matakuliah.*;

class Main {
    public static void main(String[] args) {
        System.out.println("=== SISTEM KRS BARU  

        ===");

        // validasi KRS
        KrsValidator validator = new
        Krsvalidator();
        validator.validasiPrasyarat(
            "Haruka",
            "Pemrograman Berorientasi Objek");

        System.out.println();

        // Perhitungan UKT
        UKTStrategy ukt = new UKTMBKM();

        System.out.println(
            "UKT Mahasiswa MBKM : Rp "
            + ukt.hitungUKT());

        System.out.println();

        // Cetak PDF
        PDFGenerator pdf = new PDFGenerator();
        pdf.generatePDF(
            "KRS Semester Ganjil 2026");

        System.out.println();
    }
}

```

```

        // Simpan ke Database
        Database db = new NosqlDatabase();

        KRSService service =
            new KRSService(db);

        service.simpanKRS(
            "KRS Haruka Semester Ganjil");

        System.out.println();

        // Mata Kuliah Teori
        Matakuliah teori =
            new MatakuliahTeori();

        teori.tampilkanInfo();

        System.out.println();

        // Mata Kuliah Praktikum
        MatakuliahPraktikum praktikum =
            new MatakuliahPraktikum();

        praktikum.tampilkanInfo();
        praktikum.alokasiAsistenLab();
        praktikum.cekPeralatanPraktikum();

        System.out.println();

        // Mata Kuliah KKN
        Matakuliah knn =
            new MatakuliahKKN();

        knn.tampilkanInfo();
    }
}

```

Kelas Main berfungsi sebagai driver program yang menghubungkan seluruh komponen sistem. Pada kelas ini dilakukan validasi KRS, perhitungan UKT, pencetakan PDF, penyimpanan data ke database, serta pengelolaan berbagai jenis mata kuliah.

Secara keseluruhan, rancangan sistem yang baru berhasil mengatasi berbagai permasalahan pada SistemKRSManger dengan menerapkan prinsip-prinsip SOLID. Hasilnya adalah kode yang lebih modular, mudah dipelihara, mudah dikembangkan, dan lebih siap menghadapi perubahan kebutuhan sistem di masa mendatang.

V. Kesimpulan

Berdasarkan hasil analisis dan implementasi program, dapat disimpulkan bahwa permasalahan pada sistem SIAKAD disebabkan oleh desain perangkat lunak yang kurang baik, seperti adanya kelas yang memiliki terlalu banyak tanggung jawab, penggunaan interface yang tidak sesuai kebutuhan, ketergantungan langsung terhadap database tertentu, serta logika perhitungan UKT yang sulit dikembangkan.

Untuk mengatasi permasalahan tersebut, dilakukan perancangan ulang sistem dengan menerapkan prinsip-prinsip SOLID. Single Responsibility Principle (SRP) diterapkan dengan memisahkan fungsi validasi KRS, pencetakan PDF, dan penyimpanan data ke dalam kelas yang berbeda. Open Closed Principle (OCP) diterapkan pada perhitungan UKT melalui penggunaan interface UKTStrategy sehingga penambahan jalur mahasiswa baru dapat dilakukan tanpa mengubah kode yang sudah ada. Interface Segregation Principle (ISP) diterapkan dengan memisahkan interface MataKuliah dan Praktikum sehingga setiap kelas hanya mengimplementasikan fungsi yang dibutuhkan. Dependency Inversion Principle (DIP) diterapkan melalui penggunaan interface Database sehingga sistem dapat bermigrasi dari MySQL ke NoSQL dengan mudah.

Dengan penerapan prinsip SOLID, sistem menjadi lebih modular, mudah dipelihara, mudah dikembangkan, dan lebih siap menghadapi perubahan kebutuhan di masa mendatang. Selain itu, risiko terjadinya kesalahan akibat perubahan kode dapat diminimalkan sehingga kualitas perangkat lunak menjadi lebih baik.

VI. Daftar Pustaka

Maiga, J., & Emanuel, A. W. R. (2019). Gamification for Teaching and Learning Java Programming for Beginner Students-A Review. *J. Comput.*, 14(9), 590-595.

Wiener, R., & Pinson, L. J. (2000). *Fundamentals of OOP and data structures in Java*.

Cambridge University Press. Poo, D., Kiong, D., & Ashok, S. (2007). *Object-oriented programming and Java*. Springer Science & Business Media.